

RISC-V 技术文档 Demo

葛良晨

2026 年 5 月 4 日

目录

0.1 修订历史	ii
第一章 Roadmap of Linux Kernel	1
1.1 系统使用与管理	1
1.1.1 进程与服务管理	1
附录 A 一些碎碎念	5

0.1 修订历史

1.0 葛良晨 (2025-05-4)

Added

- L^AT_EX 模版搭建完成

第一章 Roadmap of Linux Kernel

Linux 学习的 Roadmap，主要包括三大主题（系统的使用与管理，核心子系统原理，内核深度剖析），以及一个额外的高级主题。

- **系统的使用与管理**：这一部分不是简单介绍基础操作。而是比价深入的理解各种基础概念。
 - **进程与服务管理**：使用 ps/top/htop 等工具，相关的系统调用 fork/exec/wait 等；服务管理 systemd/units；Linux 的启动流程演化路径。
 - **用户/权限/安全模型**：UID/GID/权限位/ACL；sudo 机制，PAM（安全）capabilities（替代 root）；Linux 安全模块（LSM）和 SELinux/AppArmor 等安全框架；Linux 的安全加固和最佳实践。
 - **文件系统与存储管理**：VFS 抽象；常见 FS ext4/xfs/btrfs/procfs/sysfs；mount / namespace。存储上，block device；LVM；RAID；buffer vs cache；direct IO/ buffered IO；IO 调度器（CFQ/Deadline/NOOP）；
 - **网络管理**：socket 编程；TCP/IP；ss/netstat；iproute2 工具；iptables/nftables；network namespaces；
 - **系统监控与性能分析**：iostat/vmstat/pidstat；perf 工具；strace/ltrace/bpfttrace；systemtap/eBPF；日志管理（syslog/journald/dmesg）；
- **核心子系统原理**：这一部分深入探讨 Linux 内核的核心子系统，包括进程管理、内存管理、文件系统、网络协议栈等。通过理解这些子系统的原理，读者可以更好地理解 Linux 内核的工作机制。
- **内核深度剖析**：这一部分将深入分析 Linux 内核的实现细节，包括内核模块开发、调试技巧、性能优化等内容。通过实际的代码示例和案例分析，读者可以掌握内核开发和调试的技能。
- **高级主题**：这一部分将涵盖一些高级主题，如实时内核、嵌入式 Linux、安全性增强等。这些内容将帮助读者了解 Linux 在特定领域的应用和发展趋势。

1.1 系统使用与管理

1.1.1 进程与服务管理

启动流程

启动流程 Linux 的启动流程可以分为以下几个阶段：

- **BIOS/UEFI 阶段**：计算机开机后，BIOS 或 UEFI 固件会进行硬件初始化，并寻找可引导设备。找到后，加载引导加载程序（如 GRUB）到内存并执行
- **引导加载程序阶段**：引导加载程序负责加载 Linux 内核和初始 RAM 盘（initrd/initramfs）到内存，并将控制权转交给内核。

- **内核初始化阶段：**内核开始执行，进行硬件检测和初始化，挂载根文件系统，并启动第一个用户空间进程（通常是 `init` 或 `systemd`）。内核初始化完成后，内核会调用 `init` 进程，通常是通过 `execve("/sbin/init", ...)`，这样第一个用户态进程（PID 1）就开始运行。
- **用户空间初始化阶段：**`init` 或 `systemd` 负责启动系统服务和用户进程，完成系统的初始化。

展开说说内核的启动过程 Bootloader（比如 GRUB）完成的工作有：

- 加载内核映像（`vmlinuz`）和初始 RAM 盘（`initrd/initramfs`）到内存。
- 设置内核启动参数 `cmdline`（如 `root=`、`console=` 等）。
- 将控制权转交给内核，通常是通过跳转到内核入口点（如 `0x100000`）来实现。

下面就进入到内核启动流程：

1. **体系结构相关的初始化：**内核会进行 CPU 特定的初始化，建立最初的栈，如设置 GDT/IDT、启用分页、初始化中断控制器、设置 CPU 模式等。
2. **`start_kernel` 函数：**这是内核的主入口点，负责调用各种初始化函数来设置内核环境。
3. **内核子系统初始化：**内核会依次初始化内存管理、调度器、中断子系统、定时器、设备模型、驱动框架和文件系统等核心模块，并完成根文件系统的挂载准备。

为什么要挂载 `initramfs`？

如何创建 `kernel_thread` 的？内核 PID 1 是如何变成用户态的？

`initramfs->rootfs` 切换是如何进行的？

内核的概念 内核用于整个操作系统核心，用于管理资源（CPU，内存，网络，存储，系统调用等）。内核是资源的拥有者和仲裁者。指责包括：

- **进程调度：**分配 CPU 时间片，管理进程状态（就绪、运行、阻塞等）。
- **内存管理：**分配和回收内存，管理虚拟内存和物理内存。
- **文件系统：**提供统一的接口访问不同类型的文件系统。
- **网络协议栈：**处理网络通信，支持各种协议（TCP/IP 等）。
- **设备驱动：**管理硬件设备的访问和控制。
- **系统调用：**提供用户空间与内核空间的接口，允许用户程序请求内核服务。

内核的主要工作方式：事件驱动 + 被动执行 + 少量主动线程

内核的代码运行在 3 种 context 中：

- **用户空间进程：**当用户空间的程序执行时，它会通过系统调用进入内核态，内核会执行相应的内核代码来处理请求。
- **中断处理程序：**当硬件设备发出中断信号时，内核会暂停当前正在执行的代码，转而执行相应的中断处理程序来响应硬件事件。

- **内核线程**：内核可以创建自己的线程（kernel thread），这些线程在内核态运行，执行内核任务，如处理网络请求、管理内存等。

Listing 1.1: Linux 内核的核心子系统

Linux Kernel

- 进程调度（Scheduler）
- 内存管理（Memory Management）
- 虚拟文件系统（VFS）
- 设备驱动（Drivers）
- 网络子系统（Networking）
- 系统调用接口（Syscall）
- 安全机制（LSM / 权限）

内核中的执行体 假设一个单核 CPU，它每时每刻都在执行指令，那么这个指令来自于哪里呢？有这样几种可能：

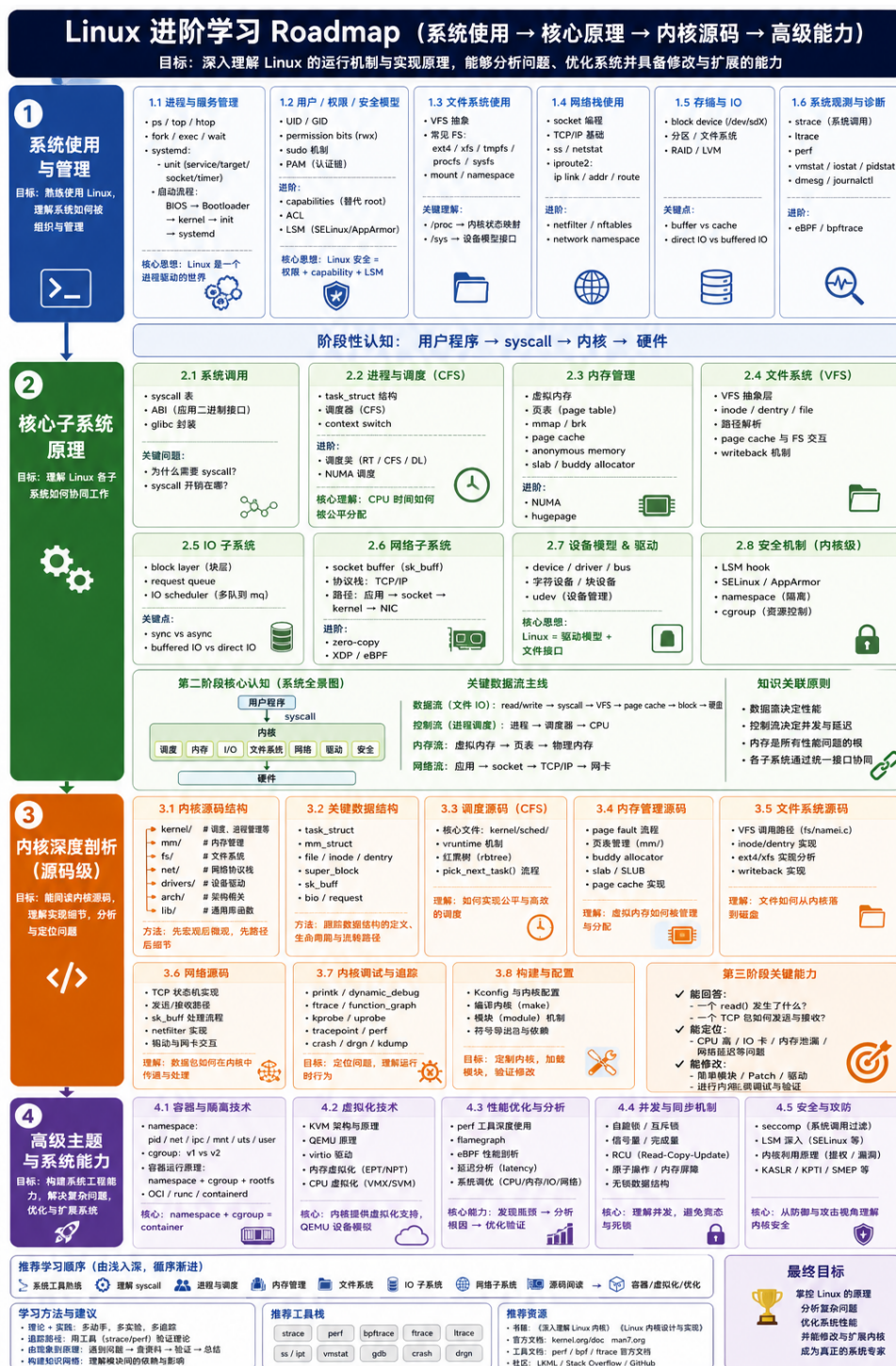
- **用户空间进程**：当用户空间的程序执行时，它会通过系统调用进入内核态，内核会执行相应的内核代码来处理请求。
- **内核 syscall**：当用户空间进程调用系统调用时，内核会执行相应的系统调用处理函数来完成用户请求。
- **内核线程**：内核可以创建自己的线程（kernel thread），这些线程在内核态运行，执行内核任务，如处理网络请求、管理内存等。
- **中断处理程序**：当硬件设备发出中断信号时，内核会暂停当前正在执行的代码，转而执行相应的中断处理程序来响应硬件事件。

现代 Linux 发行版已经从原本的 `init+etc/rc*.d` 脚本的启动方式 SysV，转变到了使用 `systemd`。启动模型也从原本串行执行脚本，变成依赖驱动的并行服务调度。

PID 1 的 `init` 进程是系统启动后第一个运行的用户空间进程，负责启动和管理其他系统服务和用户进程。它是系统的根进程，所有其他进程都是它的子孙进程。它具有如下的特点：

- **祖先进程**：所有其他进程都是 `init` 的子孙进程。当一个进程的父进程终止时，`init` 会接管这些孤儿进程。否则这些孤儿进程将成为僵尸进程，占用系统资源。
- **信号语义特殊**：`init` 进程对某些信号有特殊的处理方式。例如，`SIGTERM` 信号会被 `init` 忽略，而不是终止进程。这是为了确保系统的稳定性和可靠性。
- **系统初始化和服务管理**：`init` 负责启动和管理系统服务。比如，挂载为难系统，启动服务，设置环境。它会根据配置文件（如 `/etc/inittab` 或 `systemd` 的 `unit` 文件）来启动必要的服务，并确保它们在系统运行期间保持运行。

将内核和 PID 1 进行对比，重点是内核管理资源，PID 1 管理进程和服务。内核类似于政府 + 法律，PID 1 类似于总理，负责执行和管理系统服务，组织社会的运行。首相也不能违法，他在用户层，必须通过 `syscall` 来请求内核服务。



附录 A 一些碎碎念

i3wm 和 aerospace 是真的好用。窗口最大化直接通过快捷键完成，两个窗口并列操作也可以迅速完成。切换不同的工作区也很迅速，太完美了。简单易用，易上手，减少了许多鼠标的操作，不同费劲的用鼠标拖拽和调整大小，perfect!

真不错!